

ASE 420 - Software Design Using AI

Ways of Building Software Today

- **Classical SE:** Human-led design and implementation without generative AI assistance.
- **AI-Assisted SE:** AI copilots where human engineers prompt and review each step.
- **Automated SE:** Multi-agent systems and pipelines where AI performs multi-step tasks before human review.

| These Approaches Coexist

- These are not maturity stages to pass in order — teams use all three depending on task and risk.
- The use of automation is expanding, but the appropriate level depends on context.

| The Mission of NKU ASE

- Expanding classical SE education with AI-assisted and automated practices.
- Integrating real-world, industry-informed workflows into our curriculum.
- **For students:** Whichever mode you use, you stay responsible for requirements, decisions, and verification.

The Challenge in AI-Driven SE

- **Fast-Changing Tools:** Frameworks evolve rapidly (e.g., BMAD, OpenSpec); focus on durable concepts.
- **The "Vibe Coding" Trap:** Generating code without design can produce fragile systems when used beyond small experiments.
- **Our Strategy:** Ground AI workflows in relevant requirements, constraints, and design context.

Foundation First: Structural Grounding

- **"Structural Grounding"** is a course principle: give AI explicit context — problem, constraints, and decisions — before accepting generated code.
- Use only what fits your project's scale and risk.

Three Architecture Techniques for Grounding AI

1. **C4 Model:** Visualize system architecture at different levels of detail.
2. **ADRs (Architecture Decision Records):**
Record decision rationale and trade-offs.
3. **arc42:** Structured template for organizing architecture documentation.

1. C4 Model

| What Problems Does C4 Solve?

- Complex systems are hard to visualize with a single diagram.
- Without shared context on system boundaries, AI-generated code can drift from intended design.
- C4 offers four optional zoom levels: **Context**, **Container**, **Component**, **Code** — most teams only need the first two.

Benefits of C4 with AI

- **Context-Rich Prompting:** Sharing container and component boundaries with an LLM reduces ambiguity and drift.
- **Shared Mental Model:** Keeps developers and AI suggestions aligned on architectural layers.
- **Clear Boundaries:** Defines where each module and service lives.

C4 Model Hierarchy (Tetris Example)

Relationships:

- Player --(plays game)--> Tetris Desktop App [Python/Pygame]
- Tetris Desktop App --(reads/writes)--> Local Score Store [JSON file]

Structure:

Tetris System

- └ Container: Tetris Desktop App
 - └ Components: Game Loop, Score Manager, Board, Input Handler
- └ Container: Local Score Store

- Context and Container are a good start;
Component is useful for internal Tetris design.
- See the [official C4 guidance](#).

2. Architecture Decision Records (ADR)

| What Problems Do ADRs Solve?

- Teams frequently forget *why* a specific architecture, library, or pattern was chosen.
- AI assistants may lack complete or current project context unless decisions are stored in accessible documentation.
- An **ADR** captures a single architectural decision with its context and consequences.

Structure of an ADR

ADR-001: Strategy Pattern for Tetris Scoring

Status

Accepted

Context

Different game modes require different scoring algorithms when clearing lines.

Decision

Use the Strategy pattern for scoring instead of branching inside the main game loop.

Consequences

- Positive: Adding new scoring modes does not modify the core game loop.
- Negative: Adds extra classes to manage; unnecessary if only one scoring rule exists.

Benefits of ADRs with AI

- **Living Context:** Including ADRs in prompts aligns AI with project rules.
- **Human Decision:** AI proposes alternatives; engineers decide and document.
- **Not Automatic:** ADRs guide AI suggestions but do not enforce them.
- **Verify Compliance:** Check adherence via tests, code reviews, and architecture checks.

3. arc42 Architecture Template

| What Problems Does arc42 Solve?

- Ad-hoc architecture documentation can become incomplete or inconsistent over time.
- **arc42** is a structured, 12-section template for communicating software architecture.
- Covers requirements, quality goals, constraints, building blocks, and runtime views.

Using arc42 for a Student Project (Optional Technique)

- arc42 is an optional technique for ASE 420; you do not need all 12 sections.
- For ASE 420, selected sections are usually enough: **Context, Constraints, Building Blocks, Decisions, Quality Requirements.**
- See the [official arc42 documentation](#).

Benefits of arc42 in the AI Era

- **Structured Prompts:** Chapter headings organize context for AI drafts.
- **AI Maintenance:** LLMs propose doc updates or identify doc/code discrepancies; humans review.
- **Quality Focus:** Dedicated sections guide security and performance goals.

Monitoring Emerging AI Frameworks

- **Emerging Examples:** BMAD (AI-assisted planning and implementation workflows), OpenSpec (spec-driven development).
- **Core Principle:** Focus on **spec-driven development** rather than specific tools.
- **Approach:** Evaluate new tools, but keep structural design at the core.

One Possible AI-Assisted Design Cycle

Understand problem and constraints

- Ask AI for design alternatives and trade-offs
- Record significant decisions in an ADR
- Update C4 diagrams when boundaries change
- Implement small increment with AI assistance
- Test, review, and refine design iteratively

- An iterative cycle where design, implementation, and testing continuously feed into each other.

Example: Structurally Grounded Prompt

[Context and Architecture]

Project: Python Tetris. Component: ScoreManager (Level 3).

Decision: Follows ADR-001 (Strategy pattern for scoring).

Interface excerpt:

 @abstractmethod

 def calculate(self, lines_cleared: int, level: int) -> int: pass

[Task]

Implement `BonusScoringStrategy` using the rules below.

[Rules and Types]

- Validate types: raise `TypeError` if arguments are not `int`.
- Validate values: raise `ValueError` if `level < 1` or `lines_cleared` not in `(0, 1, 2, 3, 4)`.
- Formula: 0 lines = 0 pts; 1/2/3/4 lines = `int(base_points * level * 1.5)`
(base: 100/300/500/800; e.g., `calculate(2, 3) == 1350`, `calculate(0, 1) == 0`).

[Constraints and Deliverables]

- Implement only `BonusScoringStrategy`; do not modify `ScoreManager`, `GameBoard`, or `loop`.
- Return implementation, `pytest` suite, and brief ADR-001 adherence note.

Bridging Theory and Industry Practice

- **Structure Guides AI:** Use C4, ADRs, and arc42 as needed by project scale and risk.
- **Design Iteratively:** Define initial boundaries, then revise design based on implementation feedback.
- **Spec-Driven Workflows:** Write clear specifications; AI can assist with implementation and test generation, but engineers verify the result.

Focus on Architecture, Not Just Prompts

- A quick prompt can generate a script, but professional software requires maintainable, testable, and robust design.
- **AI proposes; engineers decide and verify.**

Protect Data When Working With AI Tools

- Only share code or data if tool policies and your organization, team, or repository permit it.
- Do not paste personal data, credentials, or proprietary code into unapproved AI tools.
- Review generated code for license risks, vulnerable dependencies, and security flaws.

| The AI-Enabled Engineer's Mindset

- **Problem First:** Focus on user needs and system constraints.
- **Iterative Design:** Sketch relevant C4 boundaries, record significant decisions in ADRs, and refine both as you build.
- **Verify Always:** Test and review code; never accept AI output on trust alone.
- **Stay Aligned:** Keep requirements, architecture, code, and tests consistent.